Multi-Networking

- 기본 쿠버네티스 Pod = 인터페이스 `eth0` 하나. 모든 트래픽이 한 경로
- **Primary network**: 모든 Pod가 갖는 기본 네트워크. Service·NetworkPolicy·DNS가 동작하는 곳
- **Secondary network**: 필요한 Pod에만 추가로 붙이는 네트워크. `net1`, `net2` ...
- 기준: Multus v4.x (thick plugin), CNI reference plugins v1.9.1

---

1. 필요성

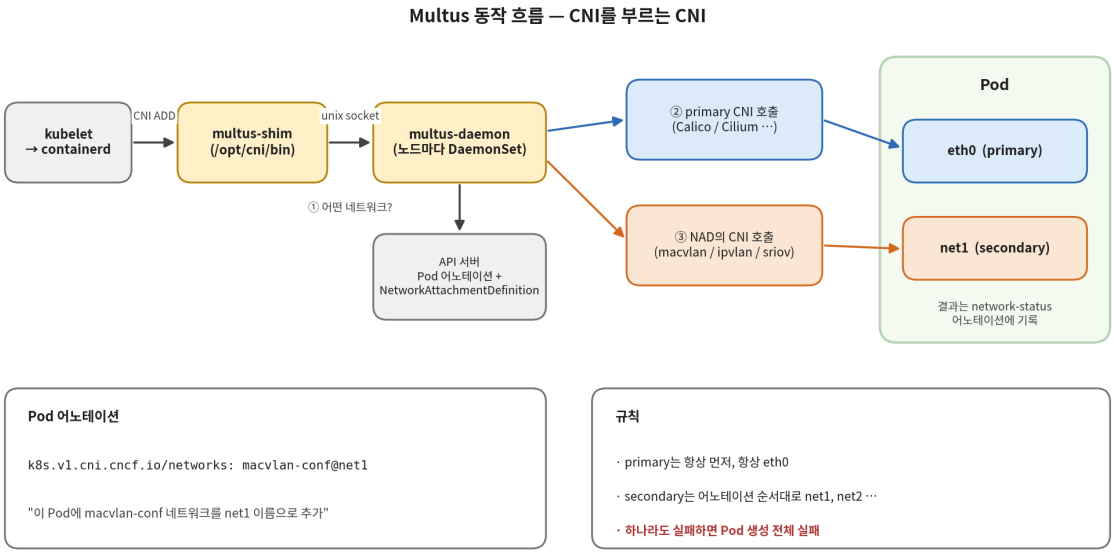
요구	예	단일 `eth0`의 한계
트래픽 분리	관리 / 데이터 / 스토리지 망 분리	모든 트래픽이 같은 경로·같은 대역폭 공유
기존 물리망 직접 연결	VLAN, 레거시 장비, 외부 L2	Pod IP는 클러스터 내부 대역
성능	5G 사용자 데이터, 고속 패킷 처리	커널 소프트웨어 경로의 오버헤드
프로토콜	멀티캐스트, L2 브로드캐스트 의존 앱	primary CNI가 지원하지 않는 경우 다수

---

2. Multus

2-1. 동작 흐름

Multus 동작 흐름



- CNI를 부르는 CNI = **meta-plugin**. 자신은 인터페이스를 만들지 않음
- 순서
  1. kubelet → containerd → `multus-shim` 실행 (CNI ADD)
  2. `multus-shim` → `multus-daemon` (unix socket)
  3. daemon이 API 서버에서 Pod 어노테이션과 NetworkAttachmentDefinition 조회
  4. primary CNI 호출 → `eth0`
  5. 어노테이션 순서대로 secondary CNI 호출 → `net1`, `net2` ...
  6. 결과를 Pod의 `k8s.v1.cni.cncf.io/network-status` 어노테이션에 기록
- **하나라도 실패하면 Pod 생성 전체 실패**

2-2. Thick 플러그인

	thin (v3 이하)	**thick (v4.0+, 권장)**
구성	바이너리 하나	`multus-shim` + `multus-daemon`
API 조회	매 호출마다 바이너리가 직접	daemon이 수행
부가 기능	—	메트릭
자원	적음	더 많음

- daemon이 Pod 안에서 delegate를 실행 → 호스트 파일이 필요한 플러그인은 `chrootDir` 또는 마운트 필요 (예: Flannel의 `/run/flannel/subnet.env`)

2-3. CNI chaining 비교

	CNI chaining	Multus
대상	**인터페이스 하나**	**인터페이스 여러 개**
구조	conflist의 플러그인을 순서대로 실행, 결과를 다음에 전달	네트워크마다 독립된 CNI 호출
예	main + `portmap` + `bandwidth`	`eth0` (Calico) + `net1` (macvlan)

3. NetworkAttachmentDefinition

- 추가 네트워크 하나 = NAD 오브젝트 하나. 내용은 그냥 **CNI 설정 JSON**

```
apiVersion: k8s.cni.cncf.io/v1
kind: NetworkAttachmentDefinition
metadata:
name: macvlan-conf
spec:
config:'{
    "cniVersion": "1.0.0",
    "type": "macvlan",
    "master": "eth1",
    "mode": "bridge",
    "ipam": { "type": "whereabouts", "range": "192.168.50.
0/24" }
  }'
```

- Pod에서 참조

```
metadata:
annotations:
k8s.v1.cni.cncf.io/networks: macvlan-conf@net1          #
텍스트 형식
    # JSON 형식: 고정 IP·MAC·default route 지정 가능
```

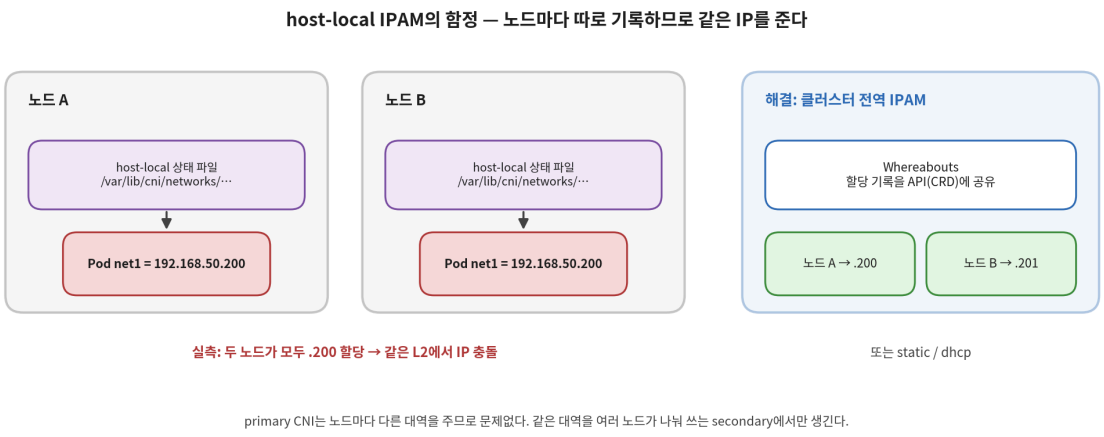
```
    # '[{"name":"macvlan-conf","ips":["192.168.50.10/24"],"default-route":["192.168.50.1"]}]'
```

- 결과 확인: `k8s.v1.cni.cncf.io/network-status`

```
[{ "name": "cbr0", "ips": ["10.244.1.73"], "default": true },
 { "name": "macvlan-conf", "interface": "net1", "ips": ["192.168.1.205"], "mac": "86:1d:96:ff:55:0d" }]
```

3-1. IPAM

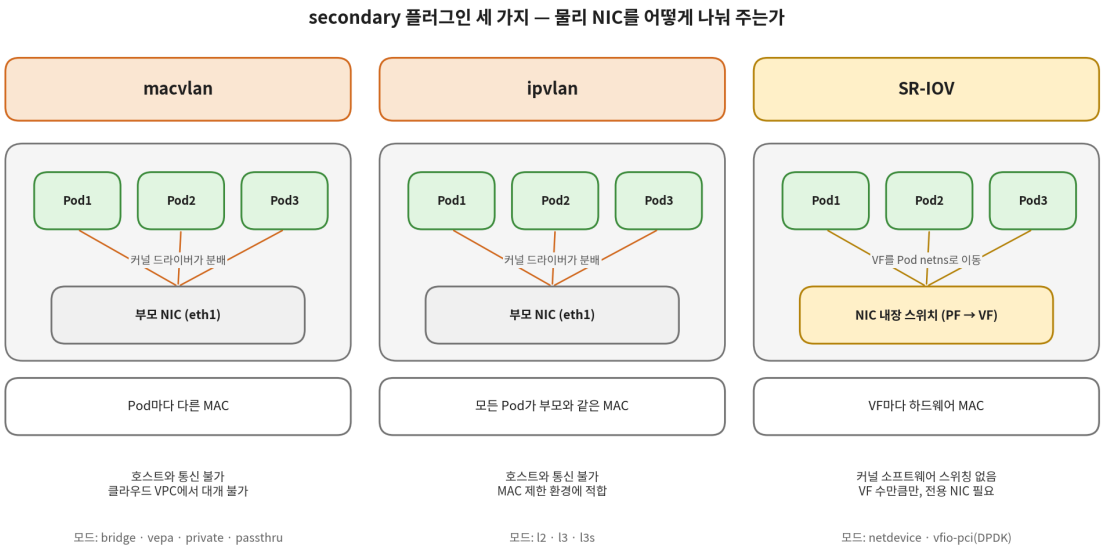
host-local IP 중복



IPAM	범위	secondary 적합성
`host-local`	**노드 하나**	여러 노드가 같은 대역 사용 시 **IP 중복**
`whereabouts`	클러스터 전역 (CRD에 기록)	권장
`static`	Pod마다 지정	소수 고정 IP
`dhcp`	외부 DHCP 서버	기존 망 정책 따를 때. ipvlan은 제약

- 실측: 노드별 host-local 상태로 같은 설정 ADD → 두 노드 모두 `192.168.50.200/24`

4. Secondary 플러그인



4-1. macvlan

- 부모 NIC 하나에 가상 NIC 여러 개. **Pod마다 고유 MAC**

모드	Pod ↔ Pod (같은 부모)	Pod ↔ 외부
bridge (기본)	직접 통신	가능
private	**차단**	가능
vepa	외부 스위치 경유 (hairpin 필요)	가능
passthru	부모 NIC를 하나의 Pod가 독점	가능

- **같은 NIC의 호스트와 통신 불가** — 커널이 부모와 자식 간 트래픽을 전달하지 않음
- 실측 (bridge 모드)

```
OK   pod1 net1 -> pod2 net1       (macvlan 간)
OK   pod1 net1 -> 외부 장비
FAIL pod1 net1 -> 호스트 phy0     (같은 부모 NIC의 호스트)
FAIL private 모드 pod -> pod
```

- 새 MAC을 허용하지 않는 환경(대부분의 퍼블릭 클라우드 VPC, 포트 보안이 걸린 스위치)에서 동작 불가

4-2. ipvlan

- 부모 NIC 하나에 가상 NIC 여러 개. **모든 Pod가 부모와 같은 MAC**

- 모드: `l2` (기본, ARP 처리) / `l3` (L3 라우팅만, 브로드캐스트·멀티캐스트 없음) / `l3s` (l3 + 호스트 netfilter 적용)
- 호스트와 통신 불가 (macvlan과 동일)
- MAC이 하나라 **MAC 제한 환경에서 사용 가능**. 퍼블릭 클라우드 VPC 사례 (Lyft `cni-ipvlan-vpc-k8s`)
- 제약: DHCP는 ClientID 필요 (CNI dhcp 플러그인 미지원)
- 같은 부모 NIC를 macvlan과 ipvlan이 동시에 쓸 수 없음

4-3. SR-IOV

- NIC 하드웨어가 물리 기능(**PF**)을 가상 기능(**VF**) 여러 개로 분할. VF 하나를 Pod에 직접 할당
- **커널 소프트웨어 스위칭 없음** — VF 간, VF↔외부 전달을 NIC가 처리

구성 요소	역할
SR-IOV Network Device Plugin	노드의 VF를 확장 리소스로 등록 (예: `intel.com/intel_sriov_netdevice`)
SR-IOV CNI	할당된 VF를 Pod netns로 이동, MAC·VLAN·IP 설정
SR-IOV Network Operator (선택)	VF 생성·드라이버 바인딩·설정 자동화

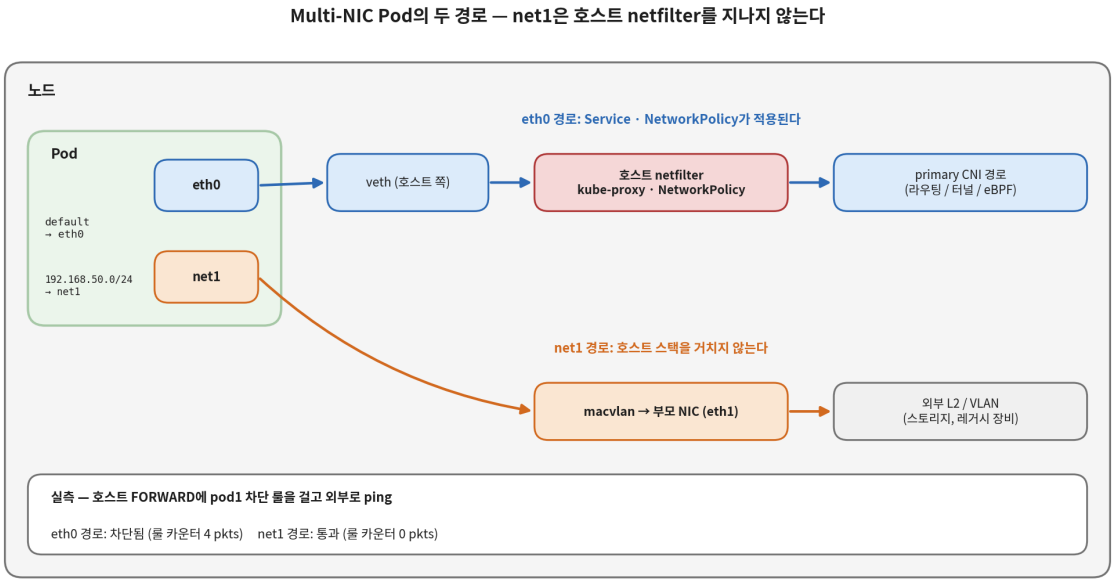
- NAD ↔ 리소스 연결

```
metadata:
annotations:
k8s.v1.cni.cncf.io/resourceName: intel.com/intel_sriov_netd
evice
```

- VF 드라이버 두 가지
  - **netdevice**: 커널 인터페이스로 보임. 일반 소켓 사용
  - **vfio-pci**: 커널에서 분리 → 사용자 공간 DPDK가 직접 처리. IP는 IPAM이 할당만 하고 인터페이스에 적용하지 않음
- 제약: VF 수 = NIC 하드웨어 한도, SR-IOV 지원 NIC 필요, 스케줄링이 노드 리소스에 묶임

---

5. Multi-NIC 패킷 흐름



- Pod 라우팅 테이블 (실측)

```
default via 10.244.0.1 dev eth0
10.244.0.0/24 dev eth0 proto kernel scope link src 10.244.0.2
192.168.50.0/24 dev net1 proto kernel scope link src 192.168.50.200
```

- 목적지에 따라 커널이 인터페이스 선택. **default는 primary**, secondary는 자기 대역만
- secondary로 default를 바꾸려면 어노테이션 `default-route` 지정
- **net1 트래픽은 호스트 netfilter를 지나지 않음** (실측)

```
호스트 FORWARD에 pod1 차단 룰 적용
FAIL pod1 eth0 -> 외부      룰 카운터 4 pkts
OK   pod1 net1 -> 외부      룰 카운터 0 pkts
```

- 결과적으로 secondary 네트워크에는

기능	적용
Service / kube-proxy	**안 됨** — ClusterIP는 primary로만
NetworkPolicy	**안 됨** — 별도 `MultiNetworkPolicy` CRD + 구현체 필요

기능	적용
클러스터 DNS	secondary IP는 Service DNS에 등록되지 않음

6. 요약

1. Multus = 다른 CNI를 차례로 부르는 meta-plugin. primary는 항상 `eth0`, 추가 네트워크는 `net1` ...
2. 추가 네트워크 = NetworkAttachmentDefinition. 내용은 CNI 설정 그대로
3. secondary IPAM은 **클러스터 전역**이어야 함. 노드별 host-local은 IP 중복
4. macvlan = MAC 여러 개, ipvlan = MAC 하나, SR-IOV = 하드웨어 분할. 앞의 둘은 **호스트와 통신 불가**
5. secondary 트래픽은 호스트 스택을 우회 → **Service·NetworkPolicy 미적용**